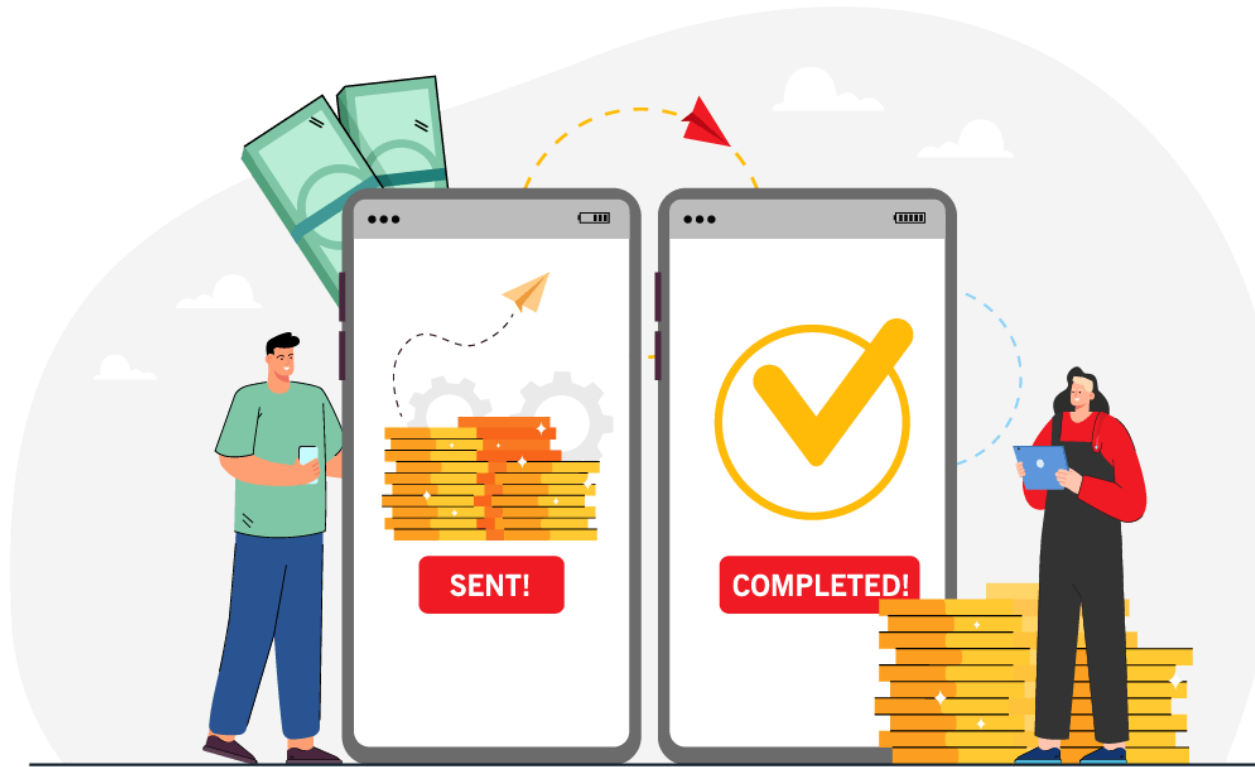




PAYOUT API SPECIFICATION

Table of Content

- 1 Business Partner Onboarding (Sales Representative will help in onboarding the partner and generating the keys for APIs).
- 2 Authorization and Authentication.
- 3 Bank List.
- 4 Purpose of Payout.
- 5 State List.
- 6 Payout to Bank Account.
- 7 Partner Callback Configuration.
- 8 API Specification Status Code.
- 9 Sample Code for Encryption and Decryption.
- 10 Escalation Matrix.

Sandbox Environment Details

Environment	Base URL
Sandbox	http://gateway.vimopay.in
Production	After audit team will share

Sandbox Credentials	
secretKey	f37701ee0778cfe1dd97b7537ae71709
saltKey	bb13602c407b1ddd6506d59e410bafeb
encryptdecryptKey	47cdba1911f078afe863774a8dffcb26
userId	E5B82667-9A9D-4A5A-A55C-F3B1E10BF370

Partner Authorization

- **URL** - /payoutapi/api/signature/authorizeuat
- **Method Type** - HTTP POST
- **Description** - Authorization API is used to check the request coming to the Vidual gateway is from the authorized partner / Merchant or not.

Headers Parameter	Type	Data Type	Description	Requisite
secretKey	Authorization	Alphanumeric	Secret Key partner will get at the time of partner's onboarding. This key is used for authorization purpose.	Mandatory
saltKey	Authorization	Alphanumeric	Salt Key partner will get at the time of partner's onboarding. This key is used for authorization purpose.	Mandatory
encryptdecryptKey	Authorization	Alphanumeric	Encrypt Decrypt Key partner will get at the time of partner's onboarding. This key is used for authorization purpose as well as to encrypt and decrypt the payload.	Mandatory
userId	Authorization	Alphanumeric	User Id, partner will get this key at the time of partner's onboarding. This key is used for authorization purpose.	Mandatory

• Postman Sample Request

Params	Authorization	Headers (11)	Body	Scripts	Tests	Settings	Cookies
☒	Connection		keep-alive				
☒	secretKey		*****				
☒	saltKey		*****				
☒	encryptdecryptKey		*****				
☒	userId		*****				
	Key		Value				Description

• Sample Response

```
{  
  "successStatus": true,  
  "message": "Success",  
  "responseCode": "000",  
  "data":  
    "7xb0GEfOmifv4lXOZa2TdS8Y2Q41slvS++I7NdFgyN4aNlVAzIZ+TKDcanuR1x1U4KT9Bg2c8p3RY5nyeOj2zkGFa7JcAKtQcrjhUETasS93B9HLr  
uXhVVr1gAZQoNFz0vQkC/WL2oKqt+prY6O+t8shtVI32/2Q76JueMprxUhZsSulWUCWpr2e6WzSfsiJryouQHBeWTf4E6tld8ZFz2QMT+PcJiq6  
3Yu3WSsAg2AlTg8MHBIZLYileELn3DgqKpakE4JQEYYUHpmom+t8ac9XMvFLfCk3iR4dc/Pa+0dtLXc0MVLtNXD/DIZuJGxv11F8fY0SrfP0dvfV  
H+LVKh5/2/N8CHyP+GZB2j7sDSptufcOMpSix7WYjZLtX2PPPlRvyukFpSTXHCIXu1+yR4XnS2FisEwYTOZ5sGtVF5FR+0JVj/Qq433BloXRXDa  
wVbbBqxkOWTdbg4UhFof8wYdm9dw6QT6S5WZKAuQjuWq+D6ynS2EBCj5iXX8tkoFnNhZPdZcfwV/7CRV8kzBHo+JCaPihXQMAXferwok9C  
/bdDH1ITipEZB/MHy1zModrhBO8P5zXZL8rXHMD"  
}
```

Bank List API

- **URL** - masterapi/api/master/banklistuat
- **Method Type** - HTTP GET
- **Description** - This API is used to get bank list that can be used to transfer the money to the beneficiary bank account using payout API.
- **Authorization Type** - Bearer Token (token value which we received from the partner authorization API call need to pass here.)

Headers Parameter	Type	Data Type	Description	Requisite
BearerToken	Authorization	Alphanumeric	token value which you received from the partner authorization API.	Mandatory
userId	Authorization	Alphanumeric	User Id, partner will get this key at the time of partner's onboarding. This key is used for authorization purpose.	Mandatory

• Postman Request to pass Header

Params	Authorization	Headers (10)	Body ●	Scripts	Tests	Settings	Cookies
☒	User-Agent	☐	PostmanRuntime/7.40.0				
☒	Accept	☐	*/*				
☒	Accept-Encoding	☐	gzip, deflate, br				
☒	Connection	☐	keep-alive				
☒	userId		*****				
☒	Authorization		Bearer KL2yN7QEeBGfzF/iuoHZV3xz9Z8YXD...				
	Key		Value				Description

- **Sample Response with Encrypted Data**

```
{  
  "successStatus": true,  
  "message": "Success",  
  "responseCode": "000",  
  "data":  
    "PTPnWGHcVnNp/YU0GX0yACWR5XWhDaxeRuS/j2Zga/yJGij+h88yDeK6b3ZzB1LzxPgj60DPoLXVpQTP4alc1WA2WAFySRTngmW0Fb  
t  
hIF/IK2MG2r/AG+4I/sbeSI98WBkq9XjAvGha0hRocZA6PbHN/xRJDWXdVtLDPdyRt8s1Ifua0hQKJBF+cxe1MWV/OqwUEzcggen7uYE+2  
hXsEa4QaLfE+zWRyFEJJGqslbxmVbeFAnx6EGkbbAr0/VEQUC3/oi19ZwZ7IB"  
}
```

• Sample Response with Decrypted Data

```
{  
  "successStatus": true,  
  "message": "Success",  
  "responseCode": "000",  
  "data": [  
    {  
      "description": "Axis Bank",  
      "code": "001"  
    },  
    {  
      "description": "Bandhan Bank",  
      "code": "002"  
    }  
  ]  
}
```

Payout Purpose API

- **URL** - /masterapi/api/master/purposelistuat
- **Method Type** - HTTP GET
- **Description** - This API is used to get purpose of payout that can be used to transfer the money to the beneficiary bank account using payout API.
- **Authorization Type** - Bearer Token (token value which we received from the partner authorization API, call need to pass here.)

Headers Parameter	Type	Data Type	Description	Requisite
BearerToken	Authorization	Alphanumeric	token value which you received from the partner authorization API.	Mandatory
userId	Authorization	Alphanumeric	User Id, partner will get this key at the time of partner's onboarding. This key is used for authorization purpose.	Mandatory

• Postman Request to pass Header

Params	Authorization	Headers (10)	Body ●	Scripts	Tests	Settings	Cookies
☒	User-Agent	☐	PostmanRuntime/7.40.0				
☒	Accept	☐	*/*				
☒	Accept-Encoding	☐	gzip, deflate, br				
☒	Connection	☐	keep-alive				
☒	userId		*****				
☒	Authorization		Bearer KL2yN7QEeBGfzF/iuoHZV3xz9Z8YXD...				
	Key		Value				Description

- Sample Response with Encrypted Data

```
{
  "successStatus": true,
  "message": "Success",
  "responseCode": "000",
  "data":
  "OgAvlcT9M+K6TtvkXYMh6PRMiEYdvt8LN/raqmSjro6shCVduPH3/NGTnWv+RrGDk7aB0JNFW1baXKJBayL14nl05Zva4+lFBmmZJxV4a6n8qChMS4HF04rXXvDklcpJ2w3ZeZ3ZrIU/ENcmrB3h/9YcxPTBTt8+k4HxBIPGBHKMc6gyl2iLClufBB9imdWVdovXq32yjAZUXAEIqfUVo18cZmhZV7RImvttkQLAuZk="
}
```

Sample Response with Decrypted Data

```
[
  {
    "description": "Cashback",
    "code": "001"
  },
  {
    "description": "Vendor Bill",
    "code": "003"
  },
  {
    "description": "Refund",
    "code": "002"
  },
  {
    "description": "Payout",
    "code": "004"
  }
]
```

Payout State List

- **URL** - /masterapi/api/master/statelistuat
- **Method Type** - HTTP GET
- **Description** - This API is used to get state list for beneficiary location for which is to be used during the payout API.
- **Authorization Type** - Bearer Token (token value which we received from the partner authorization API call need to pass here.)

Headers Parameter	Type	Data Type	Description	Requisite
Bearer Token	Authorization	Alphanumeric	token value which you received from the partner authorization API.	Mandatory
userId	Authorization	Alphanumeric	User Id, partner will get this key at the time of partner's onboarding. This key is used for authorization purpose.	Mandatory

• Postman Request to pass Header

Params	Authorization	Headers (10)	Body ●	Scripts	Tests	Settings	Cookies
☒	User-Agent	☐	PostmanRuntime/7.40.0				
☒	Accept	☐	*/*				
☒	Accept-Encoding	☐	gzip, deflate, br				
☒	Connection	☐	keep-alive				
☒	userId		*****				
☒	Authorization		Bearer KL2yN7QEeBGfzF/iuoHZV3xz9Z8YXD...				
	Key		Value				Description

• Sample Response with Encrypted Data

```
{  
  "successStatus": true,  
  "message": "Success",  
  "responseCode": "000",  
  "data": "yfgjThx0TJWuM6xyLL9/8ZX2bRgK+A8ewv0nZmsmkP6xDjl/aMuA2+EW1ilvHala6D1NWJeB6  
bAlqh1N2+GGbBqUAgd8leDF+xi/wARe8myJEZTQUj3+GQBU CXCKQXNunG1RsBKjndpJyhNAw  
ELOaeyJPmEnvrrK6zCpgEGff+7TrNYbS+Cia4eLEhYAab2/PTPnWGhCvNp/YU0GX0yACWR5XW  
hDaxeRuS/j2Zga/yJGij+h88yDeK6b3ZzB1LzxPgj60DPoLXVpQTP4alc1WA2WAFySRTngmW0Fbt  
JBhIF/IK2MG2r/AG+4I/sbeSI98WBkq9XjAvGha0hRocZA6PbHN/xRJDW XDVTLDpdyRt8s1Ifua0hQKJBF+cxe1MWV/OqwUEzcggen7uYE  
+2hXsEa4QaLfE+zWRyFEJJGqslbxmVbeFAnx6EGkbbAr0/VEQUC3/oi19ZwZ7IB"  
}
```

- Sample Response with Decrypted Data

```
{  
  "successStatus": true,  
  "message": "Success",  
  "responseCode": "000",  
  "data": [  
    {  
      "description": "Andra Pradesh",  
      "code": "AP "  
    },  
    {  
      "description": "Assam",  
      "code": "AS "  
    }  
  ]  
}
```

Payout to Bank Account

- **URL** - /payoutapi/api/payment/payoutsuat
- **Method Type** - HTTP POST
- **Description** - This API is used to send the money to the beneficiary account. For each request merchant has to pass a unique merchant ref Id. Merchant must be active to initiate any financial or non-financial transaction.
- **Authorization Type** - Bearer Token (token value which we received from the partner authorization API call need to pass here.)

Headers Parameter	Type	Data Type	Description	Requisite
Bearer Token	Authorization	Alphanumeric	token value which you received from the partner authorization API.	Mandatory
userId	Authorization	Alphanumeric	User Id, partner will get this key at the time of partner's onboarding. This key is used for authorization purpose.	Mandatory

Body Parameter	Type	Data Type	Description	Requisite
amount	Body Param	Numeric	Amount to be transfer. Amount can't be less than 100 rupees	Mandatory
merchantRefId	Body Param	Alphanumeric	Unique Reference Id for each request	Mandatory
beneficiaryBank	Body Param	Alphanumeric	Bank Code needs to be passed from bank list API. 001, 002 etc	Mandatory
paymentPurpose	Body Param	Numeric	Payment purpose you will get from pupose list API	Mandatory
paymentMode	Body Param	Alphabet	Payment Mode will be - IMPS - NEFT	Mandatory
beneficiaryAccountNumber	Body Param	Numeric	Beneficiary account number where the amount needs to be pushed.	Mandatory
beneficiaryIFSC	Body Param	Alphanumeric	Beneficiary IFSC	Mandatory
beneficiaryMobileNumber	Body Param	Numeric	Beneficiary Mobile Number	Mandatory

Body Parameter	Type	Data Type	Description	Requisite
beneficiaryLocation	Body Param	Alphanumeric	Beneficiary Location. Need to pass state code from State API	Mandatory
beneficiaryName	Body Param	Alphabet	Name of Beneficiary need to be passed	Mandatory
Lat	Body Param	Numeric	Latitude of the user from where the transaction is to happen	Mandatory
Long	Body Param	Numeric	Longitude of the user from where the transaction is to happen	Mandatory
udf1	Body Param	Alphanumeric	Merchant can pass any value for reference	Non- Mandatory
udf2	Body Param	Alphanumeric	Merchant can pass any value for reference	Non- Mandatory
udf3	Body Param	Alphanumeric	Merchant can pass any value for reference	Non- Mandatory

• Postman Request to pass Header

Params	Authorization	Headers (10)	Body ●	Scripts	Tests	Settings	Cookies
☒	User-Agent	☐	PostmanRuntime/7.40.0				
☒	Accept	☐	*/*				
☒	Accept-Encoding	☐	gzip, deflate, br				
☒	Connection	☐	keep-alive				
☒	userId		*****				
☒	Authorization		Bearer KL2yN7QEeBGfzF/iuoHZV3xz9Z8YXD...				
	Key		Value				Description

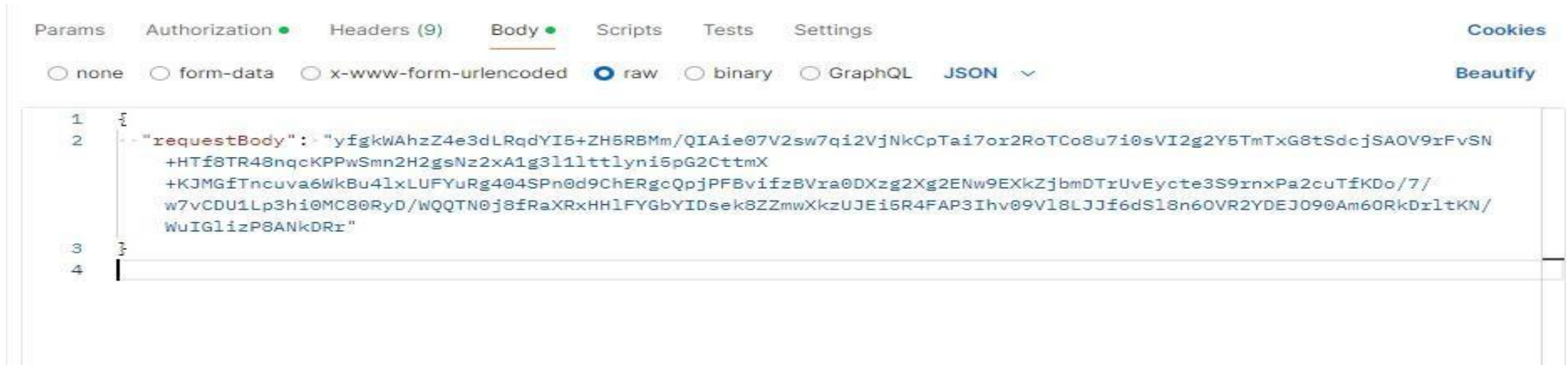
• Sample Plain Text Payload -

```
{  
  "amount": 100.0,  
  "merchantRefId": "MER123456",  
  "beneficiaryBank": "013",  
  "paymentPurpose": "004",  
  "paymentMode": "imps",  
  "beneficiaryAccountNumber": "50200030251661",  
  "beneficiaryIFSC": "HDFC0000516",  
  "beneficiaryMobileNumber": "9876543211",  
  "beneficiaryName": "Amit",  
  "beneficiaryLocation": "JH",  
  "lat": "28.7041",  
  "long": "77.1025",  
  "udf1": "",  
  "udf2": "",  
  "udf3": ""  
}
```

• Sample Plain Text Payload

```
{  
  "requestBody":  
  "Gll4gsT8Hf+xV8PofIIh6PQ30AdF5ldYZeDPqivup4783W8M5rSH5Juh2y2hF7+Dyv/His4JVU6ZZ6haYTml+Bksucfa4+lKCHnePw1rabf8  
  qHlCfMzP0duODq6sz5EVlwPBLqva+9JrVtcmrAvq/YEcXPTTUs9pisu5BN2BVWy8ElNu4t0YPoyqMRdzVmMh"  
}
```

Postman Request to pass Body Parameter



- Sample Response with Encrypted Data

```
{  
  "successStatus": true,  
  "message": "Success",  
  "responseCode": "000",  
  "data": "yfgjThx0TJWuM6xyLL9/8ZX2bRgK+A8ewv0nZmsmkP6xDjl/aMuA2+EW1ilvHala6D1NWJeB6  
bAlqh1N2+GGbBqUAgd8leDF+xi/wARe8myJEZTQUj3+GQBU CXCKQXNunG1RsBKjndpJyhNAw  
ELOaeyJPmEnvrrK6zCpgEGff+7TrNYbS+Cia4eLEhYAab2/PTPnWGHcVnp/YU0GX0yACWR5XW  
hDaxeRuS/j2Zga/yJGij+h88yDeK6b3ZzB1LzxPgj60DPoLXVpQTP4alc1WA2WAFySRTngmW0Fbt  
JBhIF/IK2MG2r/AG+4I/sbeSI98WBkq9XjAvGha0hRocZA6PbHN/xRJDWXdVtLDPdyRt8s1Ifua0hQKJBF+cxe1MWV/OqwUEzcg7uYE  
+2hXsEa4QaLFE+zWRyFEJJGqslbxmVbeFAnx6EGkbbAr0/VEQUC3/oi19ZwZ7IB"  
}
```

• Sample Response with Decrypted Data

Note: Transaction Status will be showed “queued”
txnStatusCode : 004 means transaction is queued

```
{  
  "txnStatus": "Queued",  
  "txnStatusCode": "004",  
  "responseMessage": "Your request is been queued",  
  "txnId": "25189ec9-1115-463c-ae68-d13b20d653fa",  
  "amount": "100.0",  
  "paymentMode": "imps",  
  "paymentPurpose": "004",  
  "merchantRefId": "MER123442251",  
  "beneficiaryBank": "013",  
  "beneficiaryAccountNumber": "50200030251661",  
  "beneficiaryIFSC": "HDFC0000516",  
  "beneficiaryMobileNumber": "9876543211",  
  "beneficiaryLocation": "JH",  
  "beneficiaryName": "Switch",  
  "charges": 1.0,  
  "lat": "28.7041",  
  "long": "77.1025",  
  "udf1": "",  
  "udf2": "",  
  "udf3": ""  
}
```

Partner Have to Provide Callback URL.

- **URL** - provided by the partner / Merchant
- **Method Type** - HTTP POST
- **Description** - Merchant has to share the callback URL where Vidual gateway team will post the status of the transaction (Success / Failed). By default, merchant have to keep the transaction is in progress state / pending till there is no final status of the transaction

Request parameter posted by Vidual team to merchant callback are as follows: -

Request Parameter	Type	Data Type	Description	Requisite
beneficiaryMobileNumber	Body Param	Numeric	Beneficiary Mobile Number	Mandatory
beneficiaryBank	Body Param	Numeric	Beneficiary Bank code will be returned.	Mandatory

Request Parameter	Type	Data Type	Description	Requisite
beneficiaryAccountNumber	Body Param	Numeric	Beneficiary Account Number	Mandatory
beneficiaryIFSC	Body Param	Alphanumeric	Beneficiary IFSC Number	Mandatory
beneficiaryLocation	Body Param	Alphanumeric	Beneficiary location coming from state master list Number	Mandatory
beneficiaryName	Body Param	Alphabet	Beneficiary Name	Mandatory
merchantRefId	Body Param	Alphanumeric	Unique Reference Id for each request	Mandatory
amount	Body Param	Numeric	Amount must be numeric and must be below 1 Lakh	Mandatory
charges	Body Param	Numeric	Charges must be numeric and based on the slab	Mandatory

Request Parameter	Type	Data Type	Description	Requisite
paymentMode	Body Param	Alphabet	Mode of payment via which the transaction is initiated - IMPS NEFT	Mandatory
paymentPurpose	Body Param	Numeric	- Purpose of payment code will be returned	Mandatory
udf1	Body Param	Alphanumeric	Merchant can pass any value for reference	Non- Mandatory
udf2	Body Param	Alphanumeric	Merchant can pass any value for reference	Non- Mandatory
udf3	Body Param	Alphanumeric	Merchant can pass any value for reference	Non- Mandatory
Lat	Body Param	Numeric	Latitude of the user from where the transaction is to happen	Mandatory
Long	Body Param	Numeric	Longitude of the user from where the transaction is to happen	Mandatory

Request Parameter	Type	Data Type	Description	Requisite
txnStatus	Body Param	Numeric	Status the transaction from bank side	Mandatory - success - failed - pending
txnId	Body Param	Alphanumeric	Vidual transaction Id	Conditional Mandatory only if the transaction reaches to Vidual server.
txnStatusCode	Body Param	Numeric	Status code of the txn from vidual	Success - 000 Failed -001 Inprogress - 002
rrn	Body Param	Numeric	RRN number of the transaction	Conditional Mandatory, only if the transaction is successful.

Body Parameter	Type	Data Type	Description	Requisite
responseMessage	Body Param	Alphabet	Indicates the transaction status or error, such as success, invalid account details, etc.	Mandatory

Request posted by Vidual gateway team on Merchant callback.

Params Authorization Headers (8) Body Scripts Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL JSON ▾

```
1  {
2    "txnStatus": "Success",
3    "txnStatusCode": "000",
4    "rrn": "502222116180",
5    "txnId": "07885C7E-BF7C-45FA-822C-4C1301BF5B3C",
6    "amount": 1000,
7    "paymentMode": "IMPS",
8    "paymentPurpose": "004",
9    "merchantRefId": "25347153",
10   "beneficiaryBank": "014",
11   "beneficiaryAccountNumber": "019205204134",
12   "beneficiaryIFSC": "ICIC0320492",
13   "beneficiaryMobileNumber": "9105502543",
14   "beneficiaryLocation": "UP",
15   "beneficiaryName": "Vidual",
16   "responseMessage": "Transaction successful",
17   "charges": 13.0,
18   "lat": "2.835413",
19   "long": "7.94135",
20   "udf1": "",
21   "udf2": "",
22   "udf3": ""
23 }
```

Callback Acknowledgement Response (if partner received the hit)

```
{  
  "successStatus": true,  
  "message": "Success",  
  "responseCode": "000"  
}
```

Sample Response with Decrypted Data for Success

```
{
  "successStatus": true,
  "message": "Success",
  "responseCode": "000",
  "data": {
    "txnId": "055f6fd6-ff6c-4e45-9e20-bf9846dbf70c",
    "amount": 5,
    "rrn": "435914148927",
    "paymentMode": "IMPS",
    "paymentPurpose": "004",
    "merchantRefId": "MER1234562",
    "beneficiaryBank": "013",
    "beneficiaryAccountNumber": "50200030251661",
    "beneficiaryIFSC": "HDFC0000516",
    "beneficiaryMobileNumber": "9876543211",
    "beneficiaryLocation": "JH",
    "beneficiaryName": "Amit",
    "responseMessage": "Transaction successful",
    "charges": 1.0,
    "lat": "28.7041",
    "long": "77.1025",
    "udf1": "",
    "udf2": "",
    "udf3": "",
    "txnStatus": "success",
    "txnStatusCode": "000"
  }
}
```

Sample Response with Decrypted Data for Failed

```
{
  "successStatus": true,
  "message": "Success",
  "responseCode": "000",
  "data": {
    "txnId": "055f6fd6-ff6c-4e45-9e20-bf9846dbf70c",
    "amount": 5,
    "rrn": NULL,
    "paymentMode": "IMPS",
    "paymentPurpose": "004",
    "merchantRefId": "MER1234562",
    "beneficiaryBank": "013",
    "beneficiaryAccountNumber": "50200030251661",
    "beneficiaryIFSC": "HDFC0000516",
    "beneficiaryMobileNumber": "9876543211",
    "beneficiaryLocation": "JH",
    "beneficiaryName": "Amit",
    "responseMessage": "Invalid combination of Account number and IFSC",
    "charges": 1.00,
    "lat": "28.7041",
    "long": "77.1025",
    "udf1": "",
    "udf2": "",
    "udf3": "",
    "txnStatus": "failed",
    "txnStatusCode": "001"
  }
}
```

Encryption Logic in C#

```
public static string Encrypt(string PlainText, string EDKey, string IVKey)
{
    byte[] key = Encoding.UTF8.GetBytes(EDKey); byte[] iv = Encoding.UTF8.GetBytes(IVKey);

    string sR = string.Empty;
    try
    {
        byte[] plainBytes = Encoding.UTF8.GetBytes(PlainText);
        GcmBlockCipher cipher = new GcmBlockCipher(new AesFastEngine()); AeadParameters parameters =
        new AeadParameters(new KeyParameter(key), 128, iv, null); cipher.Init(true, parameters);
        byte[] encryptedBytes =
        new byte[cipher.GetOutputSize(plainBytes.Length)];
        Int32 retLen = cipher.ProcessBytes
        (plainBytes, 0, plainBytes.Length, encryptedBytes, 0); cipher.DoFinal(encryptedBytes, retLen);
        sR = Convert.ToBase64String
        (encryptedBytes, Base64FormattingOptions.None);
    }
    catch (Exception ex)
    {
        Console.WriteLine(ex.Message); Console.WriteLine(ex.StackTrace);
    }
    return sR;
}
```

Decryption Logic in C#

```
public static string Decrypt(string EncryptedText, string EDKey, string IVKey)
{
    byte[] key = Encoding.UTF8.GetBytes(EDKey); byte[] iv = Encoding.UTF8.GetBytes(IVKey); string sR
    = string.Empty;
    try
    {
        byte[] encryptedBytes = Convert.FromBase64String(EncryptedText);
        GcmBlockCipher cipher = new GcmBlockCipher(new AesFastEngine()); AeadParameters parameters =
        new AeadParameters(new KeyParameter(key), 128, iv, null);
        cipher.Init(false, parameters); byte[] plainBytes =
        new byte[cipher.GetOutputSize(encryptedBytes.Length)];
        Int32 retLen = cipher.ProcessBytes
        (encryptedBytes, 0, encryptedBytes.Length, plainBytes, 0); cipher.DoFinal(plainBytes, retLen);
        sR = Encoding.UTF8.GetString(plainBytes).TrimEnd("\r\n\0".ToCharArray());
    }
    catch (Exception ex)
    {
        Console.WriteLine(ex.Message); Console.WriteLine(ex.StackTrace);
    }

    return sR;
}
```

Encryption Logic in Python

Note: Install pycryptodome & cryptography

```
from Crypto.Cipher import AES
import base64
import logging

# Set up logging
logging.basicConfig(level=logging.ERROR)

# Provided keys
edKey = "8086289ecab4a664c8410a1e79d19090"
ivKey = "7cca987fb6cfeea37022977bde6e62c0"

def encrypt(plainText):
    key = edKey.encode('utf-8')
    iv = ivKey.encode('utf-8')
    encryptedText = None

    try:
        plainBytes = plainText.encode('utf-8')

        if len(key) not in (16, 24, 32):
            raise ValueError("Key length must be 16, 24, or 32 bytes for AES.")
```

Encryption Logic in Python

```
# Note: GCM mode typically expects a 12-byte nonce.
cipher = AES.new(key, AES.MODE_GCM, nonce=iv)

encryptedBytes, tag = cipher.encrypt_and_digest(plainBytes)

# Concatenate the encrypted bytes and the tag
encryptedBytesWithTag = encryptedBytes + tag

# Encode to Base64 to get a string representation
encryptedText = base64.b64encode(encryptedBytesWithTag).decode('utf-8')

except Exception as e:
    logging.error("Encrypt error: {}".format(e))

return encryptedText

# The text you provided
plaintext = "Enter the text which you want to encrypt"

# Encrypt the provided text
encrypted_text = encrypt(plaintext)
print("Encrypted text:")
print(encrypted_text)
```

Decryption Logic in Python

Note: Install pycryptodome & cryptography

```
from Crypto.Cipher import AES
import base64
import logging

# Set up logging
logging.basicConfig(level=logging.ERROR)

# Provided keys
edKey = "8086289ecab4a664c8410a1e79d19090"
ivKey = "7cca987fb6cfeea37022977bde6e62c0"

def decrypt(encryptedText):
    key = edKey.encode('utf-8')
    iv = ivKey.encode('utf-8')
    decryptedText = None

    try:
        encryptedBytesWithTag = base64.b64decode(encryptedText)

        if len(key) not in (16, 24, 32):
            raise ValueError("Key length must be 16, 24, or 32 bytes for AES.")
```

Decryption Logic in Python

```
# Split the encrypted bytes and the tag
encryptedBytes = encryptedBytesWithTag[:-16]
tag = encryptedBytesWithTag[-16:]

cipher = AES.new(key, AES.MODE_GCM, nonce=iv)

# Decrypt and verify the tag
plainBytes = cipher.decrypt_and_verify(encryptedBytes, tag)

decryptedText = plainBytes.decode('utf-8').strip()

except Exception as e:
    logging.error("Decrypt error: {}".format(e))

return decryptedText

# The encrypted text (replace with your actual encrypted string)
encrypted_text = 'Enter the Encrypted text here'

# Decrypt the provided text
decrypted_text = decrypt(encrypted_text)
print("Decrypted text:")
print(decrypted_text)
```

Encryption Logic in php

```
<?php
// Provided keys
$edKey = "8086289ecab4a664c8410a1e79d19090";
$ivKey = "7cca987fb6cfeea37022977bde6e62c0";

// Convert keys to binary format
$key = utf8_encode($edKey);
$iv = utf8_encode($ivKey);

function encrypt($plainText, $key, $iv) {
    $encryptedText = null;

    try {
        // Check key length
        $keyLength = strlen($key);
        if (!in_array($keyLength, [16, 24, 32])) {
            throw new Exception("Key length must be 16, 24, or 32 bytes for AES.");
        }

        // Encryption
        $cipher = "aes-".($keyLength * 8)."-gcm";
```

Encryption Logic in php

```
// Encrypt the plaintext
$tag = "";
$encryptedBytes = openssl_encrypt(
    $plainText,
    $cipher,
    $key,
    OPENSSL_RAW_DATA,
    $iv,
    $tag,
    "",
    16 // Tag length
);

if ($encryptedBytes === false) {
    throw new Exception("Encryption failed: " . openssl_error_string());
}

// Concatenate the encrypted bytes and the tag
$encryptedBytesWithTag = $encryptedBytes . $tag;

// Encode to Base64 to get a string representation
$encryptedText = base64_encode($encryptedBytesWithTag);

} catch (Exception $e) {
    error_log("Encrypt error: " . $e->getMessage());
}

return $encryptedText;
43 }
```

Encryption Logic in php

```
function decrypt($encryptedText, $key, $iv) {
    $decryptedText = null;

    try {
        // Check key length
        $keyLength = strlen($key);
        if (!in_array($keyLength, [16, 24, 32])) {
            throw new Exception("Key length must be 16, 24, or 32 bytes for AES.");
        }

        // Decryption
        $cipher = "aes-".($keyLength * 8)."-gcm";

        // Decode from Base64
        $encryptedBytesWithTag = base64_decode($encryptedText);

        // Split the encrypted bytes and the tag
        $tagLength = 16;
        $encryptedBytes = substr($encryptedBytesWithTag, 0, -$tagLength);
        $tag = substr($encryptedBytesWithTag, -$tagLength);

        // Decrypt the ciphertext
        $decryptedBytes = openssl_decrypt(
            $encryptedBytes,
            $cipher,
            $key,
            OPENSSL_RAW_DATA,
            $iv,
            $tag,
            ""
        );
    }
}
```

Encryption Logic in php

```
if ($decryptedBytes === false) {  
    throw new Exception("Decryption failed: " . openssl_error_string());  
}  
  
$decryptedText = $decryptedBytes;  
  
} catch (Exception $e) {  
    error_log("Decrypt error: " . $e->getMessage());  
}  
  
return $decryptedText;  
}  
  
// The text you provided  
$plaintext = 'Enter the text you want to Encrypt';  
  
// Encrypt the provided text  
$encrypted_text = encrypt($plaintext, $key, $iv);  
echo "Encrypted text:\n";  
echo $encrypted_text;  
  
// Optionally, decrypt to verify  
//$decrypted_text = decrypt($encrypted_text, $key, $iv);  
//echo "\n\nDecrypted text:\n";  
//echo $decrypted_text;  
?>
```

Decryption Logic in php

```
<?php
// Provided keys
$edKey = "8086289ecab4a664c8410a1e79d19090";
$ivKey = "7cca987fb6cfeea37022977bde6e62c0";

// Convert keys to binary format
$key = utf8_encode($edKey);
$iv = utf8_encode($ivKey);

function decrypt($encryptedText, $key, $iv) {
    $decryptedText = null;

    try {
        // Check key length
        $keyLength = strlen($key);
        if (!in_array($keyLength, [16, 24, 32])) {
            throw new Exception("Key length must be 16, 24, or 32 bytes for AES.");
        }

        // Decryption
        $cipher = "aes-".($keyLength * 8)."-gcm";

        // Decode from Base64
        $encryptedBytesWithTag = base64_decode($encryptedText);
```

Decryption Logic in php

```
// Split the encrypted bytes and the tag
$tagLength = 16;
$encryptedBytes = substr($encryptedBytesWithTag, 0, -$tagLength);
$tag = substr($encryptedBytesWithTag, -$tagLength);

// Decrypt the ciphertext
$decryptedBytes = openssl_decrypt(
    $encryptedBytes,
    $cipher,
    $key,
    OPENSSL_RAW_DATA,
    $iv,
    $tag,
    ""
);

if ($decryptedBytes === false) {
    throw new Exception("Decryption failed: " . openssl_error_string());
}

$decryptedText = $decryptedBytes;

} catch (Exception $e) {
    error_log("Decrypt error: " . $e->getMessage());
}

return $decryptedText;
}
```

Decryption Logic in php

```
// The encrypted text (replace with your actual encrypted string)
$encrypted_text = 'Enter your encrypted text here';

// Decrypt the provided text
$decrypted_text = decrypt($encrypted_text, $key, $iv);
echo "Decrypted text:\n";
echo $decrypted_text;
?>
```

Encryption Logic in Java

```
import javax.crypto.Cipher;
import javax.crypto.spec.GCMParameterSpec;
import javax.crypto.spec.SecretKeySpec;
import java.util.Base64;

public class AESEncryption {

    private static final String edKey = "8086289ecab4a664c8410a1e79d19090";
    private static final String ivKey = "7cca987fb6cfeea37022977bde6e62c0";

    public static String encrypt(String plainText) {
        String encryptedText = null;
        try {
            byte[] key = edKey.getBytes("UTF-8");
            byte[] iv = ivKey.getBytes("UTF-8");

            if (key.length != 16 && key.length != 24 && key.length != 32) {
                throw new IllegalArgumentException("Key length must be 16, 24, or 32 bytes.");
            }
        }
    }
}
```

```

SecretKeySpec keySpec = new SecretKeySpec(key, "AES");

GCMParameterSpec gcmParameterSpec = new GCMParameterSpec(128, iv); // 128-bit authentication tag length

Cipher cipher = Cipher.getInstance("AES/GCM/NoPadding");

cipher.init(Cipher.ENCRYPT_MODE, keySpec, gcmParameterSpec);

byte[] encryptedBytes = cipher.doFinal(plainText.getBytes("UTF-8"));

encryptedText = Base64.getEncoder().encodeToString(encryptedBytes);

} catch (Exception e) {
    e.printStackTrace();
    System.err.println("Encrypt error: " + e.getMessage());
}

return encryptedText;
}

public static void main(String[] args) {
    String plaintext = "{\n" +
        "  \"userMobileNo\": \"9422811530\",\n" +
        "  \"merchantRefId\": \"Pronoy\",\n" +
        "  \"amount\": \"1\",\n" +
        "  \"lat\": \"19.251\",\n" +
        "  \"long\": \"72.234\",\n" +
        "  \"udf1\": \"\",\n" +
        "  \"udf2\": \"\",\n" +
        "  \"udf3\": \"\"\n" +
        "}";

    // Encrypt the plaintext
    String encryptedText = encrypt(plaintext);
    System.out.println("Encrypted text:");
    System.out.println(encryptedText);
}
}

```

Decryption Logic in Java

```
import javax.crypto.Cipher;
import javax.crypto.spec.GCMParameterSpec;
import javax.crypto.spec.SecretKeySpec;
import java.util.Base64;

public class AESDecryption {

    private static final String edKey = "8086289ecab4a664c8410a1e79d19090";
    private static final String ivKey = "7cca987fb6cfeea37022977bde6e62c0";

    public static String decrypt(String encryptedText) {
        String decryptedText = null;
        try {
            byte[] key = edKey.getBytes("UTF-8");
            byte[] iv = ivKey.getBytes("UTF-8");

            if (key.length != 16 && key.length != 24 && key.length != 32) {
                throw new IllegalArgumentException("Key length must be 16, 24, or 32 bytes.");
            }

            SecretKeySpec keySpec = new SecretKeySpec(key, "AES");

            GCMParameterSpec gcmParameterSpec = new GCMParameterSpec(128, iv); // 128-bit authentication tag length

            Cipher cipher = Cipher.getInstance("AES/GCM/NoPadding");

            cipher.init(Cipher.DECRYPT_MODE, keySpec, gcmParameterSpec);

            byte[] encryptedBytes = Base64.getDecoder().decode(encryptedText);

            byte[] decryptedBytes = cipher.doFinal(encryptedBytes);

            decryptedText = new String(decryptedBytes, "UTF-8");

        }
    }
}
```

Decryption Logic in Java

```
catch (Exception e) {  
    e.printStackTrace();  
    System.err.println("Decrypt error: " + e.getMessage());  
}  
  
    return decryptedText;  
}  
  
public static void main(String[] args) {  
    // Replace this with the encrypted text obtained from the encryption code  
    String encryptedText =  
    "GnEt0YGucuWgW93AXY9qvrNBhhdP/JxRaOrEsDb9/9i+nCtnqOD1od+F2n34XbOfjpSQ37VPW1bYKJddYCPouxkxg9fY7+s  
    ECGCTcEEueaPvqGRCHKmBlmUDvX1IYEclQ3SYuGO/8FzWP8qrl6uud9RkLHTRM0+2Z+kFNSXVT30OvhmaiWQTteMdk  
    Q71ILNXofV6DL0nEAITBMvVfVK0nj4I7H+zYO7S+hROqEPWCwoq6Xbt6+pwZLu0oFtZa9Q9w==";  
  
    // Decrypt the ciphertext  
    String decryptedText = decrypt(encryptedText);  
    System.out.println("Decrypted text:");  
    System.out.println(decryptedText);  
}  
}
```

```
public class EncryptUtils {

    private static final String edKey = "8086289ecab4a664c8410a1e79d19090";
    private static final String ivKey = "7cca987fb6cfeea37022977bde6e62c0";

    public static String encrypt(String plainText) {
        String encryptedText = null;
        try {
            byte[] key = edKey.getBytes(StandardCharsets.UTF_8);
            byte[] iv = ivKey.getBytes(StandardCharsets.UTF_8);
            byte[] plainBytes = plainText.getBytes(StandardCharsets.UTF_8);

            if (key.length != 16 && key.length != 24 && key.length != 32) {
                throw new IllegalArgumentException("Key length must be 16, 24, or 32 bytes for AES.");
            }

            // if (iv.length != 12) {
            //     throw new IllegalArgumentException("IV length must be 12 bytes for GCM mode.");
            // }

            SecretKeySpec secretKeySpec = new SecretKeySpec(key, "AES");
            GCMParameterSpec gcmSpec = new GCMParameterSpec(128, iv);

            Cipher cipher = Cipher.getInstance("AES/GCM/NoPadding");
            cipher.init(Cipher.ENCRYPT_MODE, secretKeySpec, gcmSpec);

            byte[] encryptedBytes = cipher.doFinal(plainBytes);

            encryptedText = Base64.encodeToString(encryptedBytes, Base64.NO_WRAP);
        } catch (Exception e) {
            e.printStackTrace();
            return "";
        }

        return encryptedText;
    }
}
```

```
public class DecryptUtils {

    private static final String edKey = "8086289ecab4a664c8410a1e79d19090";
    private static final String ivKey = "7cca987fb6cfeea37022977bde6e62c0";

    public static String decrypt(String encryptedText) {
        String decryptedText = null;

        try {
            byte[] key = edKey.getBytes(StandardCharsets.UTF_8);
            byte[] iv = ivKey.getBytes(StandardCharsets.UTF_8);
            byte[] encryptedBytes = Base64.decode(encryptedText, Base64.NO_WRAP);

            if (key.length != 16 && key.length != 24 && key.length != 32) {
                throw new IllegalArgumentException("Key length must be 16, 24, or 32 bytes for AES.");
            }

            // if (iv.length != 12) {
            //     throw new IllegalArgumentException("IV length must be 12 bytes for GCM mode.");
            // }

            SecretKeySpec secretKeySpec = new SecretKeySpec(key, "AES");
            GCMParameterSpec gcmSpec = new GCMParameterSpec(128, iv);

            Cipher cipher = Cipher.getInstance("AES/GCM/NoPadding");
            cipher.init(Cipher.DECRYPT_MODE, secretKeySpec, gcmSpec);

            byte[] decryptedBytes = cipher.doFinal(encryptedBytes);

            decryptedText = new String(decryptedBytes, StandardCharsets.UTF_8);
        } catch (Exception e) {
            e.printStackTrace();
        }

        return decryptedText;
    }
}
```

Status Code for API

S. No	Status Code	Description
1	000	Success
2	001	Failed
3	002	Pending
4	003	Validation Failed
5	004	Queued

Escalation Matrix

Level	Name	Email
Level 1	Pronoy	pronoy.bhattacharya@vidual.in
Level 2	Tekchand	tekchand.sharma@vidual.in

*Thank
you!*